UNIVERSITAS MIKROSKIL | Fakultas INFORMATIKA

Ensemble Learning, Validation Strategy, and Imbalanced Data

IF2116 - Pembelajaran Mesin

Pertemuan 5



Sarjana Strata 1

Teknik Informatika

Materi Pertemuan 4

- **a** Intuisi Ensemble & Bias–Variance
- **b** Bagging & Random Forest
- **c** Boosting: AdaBoost, GBM, XGBoost
- **d** Voting & Stacking
- **e** Strategi Validasi: Hold-out & Leakage
- **f** Cross-Validation & Tuning
- **g** Data Imbalanced: Masalah & Metrik
- **h** Data Imbalanced: Teknik Penanganan

Bekal dari Pertemuan 3

Underfitting vs Overfitting (bias–variance), regularisasi Ridge/Lasso, metrik evaluasi regresi, dan baseline model. Hari ini semuanya dipakai untuk **meningkatkan** dan **mempertanggungjawabkan** model baseline project.

Tujuan Utama Pembelajaran

- Menjelaskan **mengapa** menggabungkan banyak model (ensemble) dapat menurunkan error, serta membedakan *Bagging, Random Forest, Boosting, Voting,* dan *Stacking* dari cara kerja dan masalah yang diatasinya (variance vs bias).

- Merancang **strategi validasi yang jujur**: memisahkan train/validation/test, memilih *Cross-Validation* yang sesuai karakter data (Stratified, Group, Time Series), melakukan *hyperparameter tuning*, dan mencegah *data leakage*.

- Mengenali masalah **data tidak seimbang (imbalanced)**, memahami mengapa *accuracy* menyesatkan, dan memilih metrik yang tepat (Precision, Recall, F1, PR-AUC).

- Menerapkan teknik penanganan imbalanced (*class weight, resampling/SMOTE, threshold moving*) di dalam pipeline yang benar, lalu menginterpretasikan hasilnya secara kuantitatif.

Mampu **membangun**, **memvalidasi**, dan **menyeimbangkan** model prediksi agar hasilnya **akurat sekaligus dapat dipercaya**.

Dari Baseline ke Model yang Layak Dipercaya

1 Ensemble Learning

Bagaimana membuat model lebih kuat?

Gabungkan banyak model agar lebih akurat dan stabil. Menyerang dua sumber error: **variance** dan **bias**.

2 Validation Strategy

Bagaimana tahu model benar-benar bagus?

Ukur performa pada data yang tidak pernah "dilihat" model. Hindari ilusi akurasi dari **leakage** dan **overfitting ke test set**.

3 Imbalanced Data

Bagaimana jika kelas sangat timpang?

Pilih metrik dan teknik agar **kelas minoritas** (fraud, penyakit, cacat) tidak diabaikan oleh model.

Pertemuan 3 Baseline & evaluasi awal → **Pertemuan 4** Pengembangan model & akurasi → **UTS** Penyelesaian Milestone 1

Materi hari ini adalah bahan utama tahap **pengembangan model** pada project milestone kelompok.

UNIVERSITAS
MIKROSKIL

Ensemble Learning

Intuisi • Bagging & Random Forest • Boosting • Voting & Stacking

a ENSEMBLE LEARNING

Ide Dasar: Kebijaksanaan Orang Banyak

Ensemble = beberapa model dasar (base learner) digabung menjadi satu prediksi final.

> *"...you will find that this aggregated answer is better than an expert's answer. This is called the wisdom of the crowd."*
>
> — A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow (bab Ensemble Learning)

Analogi: Panel Dokter

Satu dokter bisa salah diagnosis. Lima dokter dari spesialisasi berbeda yang berdiskusi lalu mengambil keputusan bersama jauh lebih jarang salah — **dengan syarat** mereka tidak membuat kesalahan yang sama.

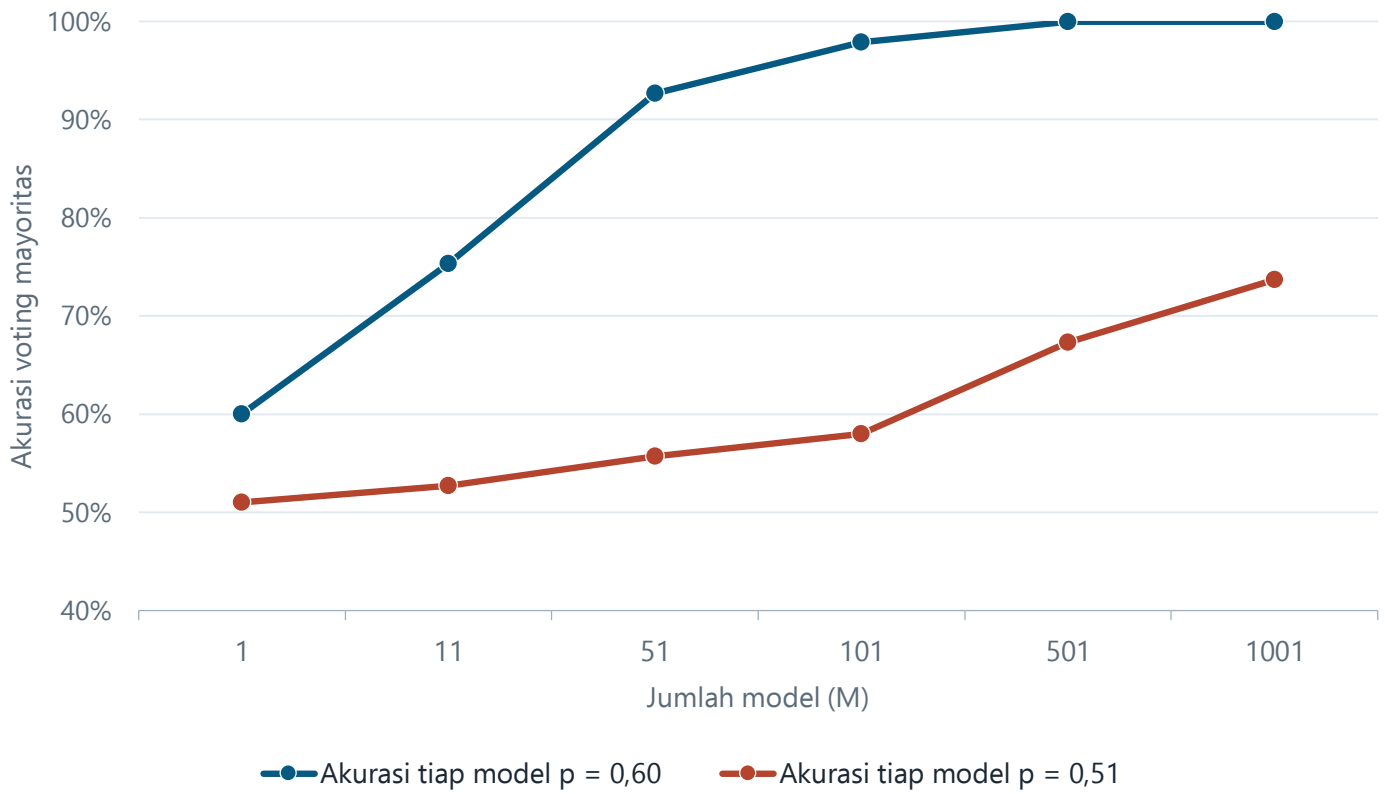
Tiga Syarat Ensemble yang Efektif

- **Akurat**: tiap model lebih baik dari menebak acak.
- **Beragam**: kesalahan antar model tidak seragam (diversity).
- **Teragregasi**: digabung lewat voting atau rata-rata.

Regresi: $\hat{y} = (1/M) \Sigma \hat{y}_m(x)$ **Klasifikasi:** $\hat{y} = \text{modus}\{\hat{y}_1, ..., \hat{y}_m\}$ (hard voting)

Mengapa Ensemble Bekerja? Hitung Saja

Voting mayoritas dari M model independen, masing-masing hanya sedikit lebih baik dari tebakan acak.



97,9%

M = 101, p = 0,60

58,0%

M = 101, p = 0,51

$$P(\text{benar}) = \Sigma\, C(M,k)\, p^k\, (1-p)^{M-k},\quad k > M/2$$

Catatan penting: hasil di atas mengasumsikan kesalahan antar model **independen**.

Di praktik model saling berkorelasi → keuntungan lebih kecil. **Diversifikasi** adalah inti ensemble.

a ENSEMBLE LEARNING

Bias–Variance: Ensemble Menyerang yang Mana?

Menghubungkan Underfitting/Overfitting (Pertemuan 3) dengan strategi ensemble.

Error = Bias^2 + Variance + Noise (tak dapat dihilangkan)

Menurunkan VARIANCE

Bagging & Random Forest

- Base learner: pohon **dalam** (low bias, high variance).
- Rata-rata banyak model → fluktuasi saling meniadakan.
- Bias hampir tidak berubah.

Menurunkan BIAS

Boosting

- Base learner: pohon **dangkal** (high bias, low variance).
- Ditambah satu per satu untuk memperbaiki kesalahan.
- Terlalu banyak ronde → variance naik (overfit).

Memadukan model berbeda

Voting & Stacking

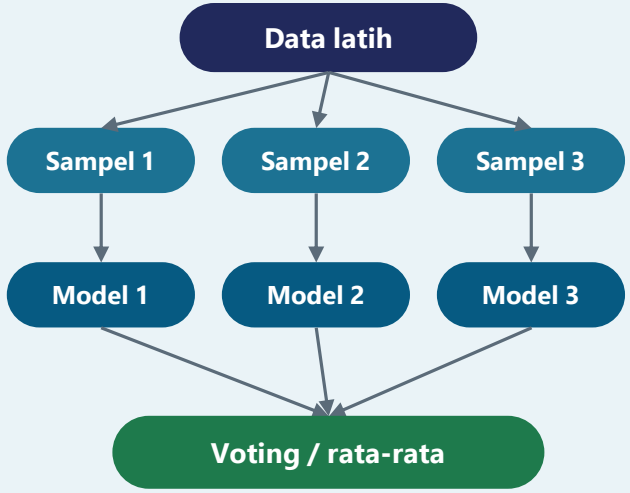
- Model berbeda jenis membuat kesalahan berbeda.
- Meta-learner belajar kapan percaya model mana.
- Manfaat bergantung pada keragaman model.

a ENSEMBLE LEARNING

Peta Metode Ensemble

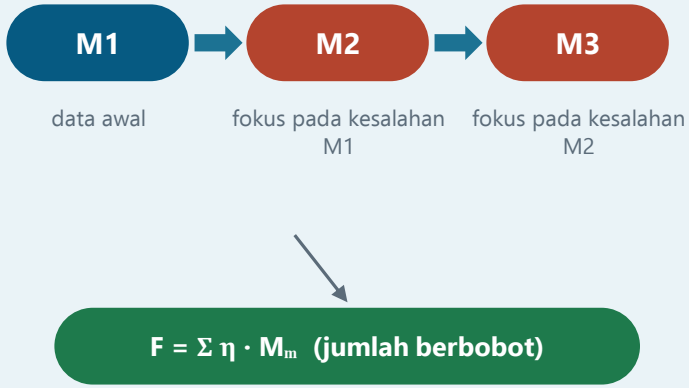
Tiga cara utama menggabungkan model.

Bagging



paralel • menurunkan variance

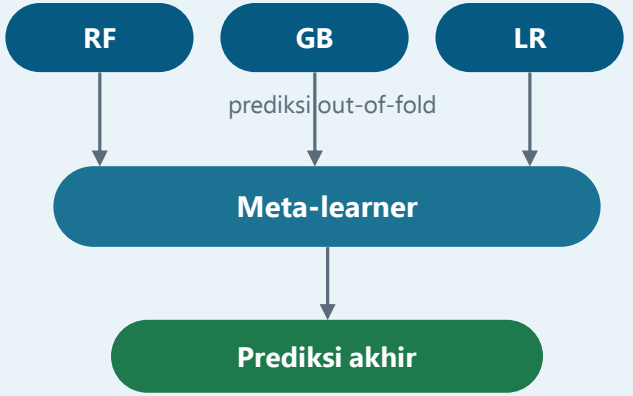
Boosting



Tiap model baru belajar dari **sisa kesalahan** gabungan model sebelumnya.

sekuensial • menurunkan bias

Stacking



Meta-learner belajar menggabungkan keluaran model dasar yang **berbeda jenis**.

berlapis • meta-learner

Bagging: Bootstrap Aggregating

Latih banyak model pada sampel acak berbeda dari data yang sama, lalu gabungkan.

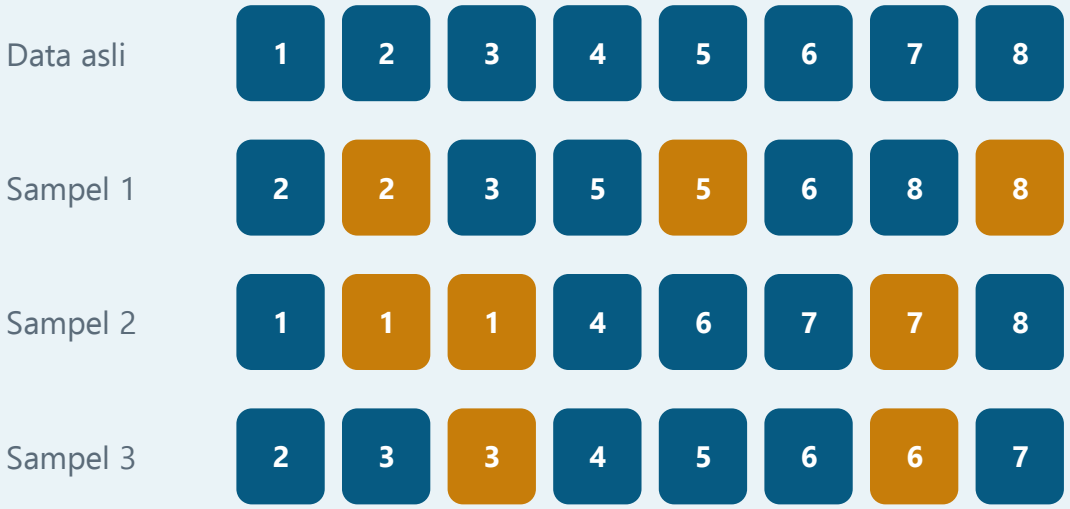
1 Bootstrap
Ambil B sampel berukuran n **dengan pengembalian** dari data latih.

2 Latih
Satu model (umumnya pohon keputusan dalam) per sampel, secara paralel.

3 Agregasi
Rata-rata prediksi (regresi) atau voting mayoritas (klasifikasi).

Hasil: **variance turun**, bias nyaris tetap. Efektif untuk model yang tidak stabil (pohon dalam).

Ilustrasi bootstrap (n = 8 data)



Biru = pertama kali terambil, oranye = duplikat. Rata-rata hanya ~63% data unik per sampel; sisanya (OOB) tidak dilihat pohon tersebut.

Bootstrap & Out-of-Bag (OOB): Validasi Gratis

Setiap pohon hanya melihat sekitar dua pertiga data; sisanya dapat dipakai menguji pohon itu.

$$P(\text{data tidak terpilih}) = (1 - 1/n)^n \to e^{-1} \approx 0{,}368$$

63,2%
data unik per sampel bootstrap

36,8%
data OOB (tidak terpilih)

Skor OOB: tiap data diprediksi hanya oleh pohon yang tidak pernah melihatnya, lalu dihitung skornya. Hasilnya estimasi generalisasi **tanpa** perlu validation set terpisah.

Notebook Pertemuan 4 – California Housing

```
rf = RandomForestRegressor(
    n_estimators=300, oob_score=True,
    random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)

print(rf.oob_score_)       # R2 OOB
print(rf.score(X_test, y_test))
```

0,811
R^2 OOB (data latih)

0,806
R^2 test set

Dua angka nyaris sama → OOB adalah estimator yang bisa dipercaya untuk Random Forest.

Random Forest = Bagging + Acak Fitur

Pada tiap percabangan, pohon hanya boleh memilih dari subset fitur acak → pohon makin tidak seragam.

> *"The Random Forest algorithm introduces extra randomness when growing trees; instead of searching for the very best feature when splitting a node, it searches for the best feature among a random subset of features."*
>
> — A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow

Variance rata-rata B pohon (korelasi ρ)

$$\text{Var} = \rho\sigma^2 + (1-\rho)\sigma^2 / B$$

- Menambah B hanya mengecilkan suku kedua.
- Suku pertama ($\rho\sigma^2$) adalah **lantai**: hanya turun jika pohon dibuat **kurang berkorelasi**.
- Subset fitur acak menurunkan ρ.

Hyperparameter	Fungsi
n_estimators	Jumlah pohon; ↑ = lebih stabil, tidak overfit
max_features	Jumlah fitur kandidat per split (kunci diversitas)
max_depth / min_samples_leaf	Mengatur kompleksitas tiap pohon
oob_score	Aktifkan estimasi generalisasi OOB

b

STUDI KASUS NYATA

BAGGING & RANDOM FOREST

Dari Satu Pohon ke Satu Hutan

Notebook Pertemuan 4 – California Housing (20.640 sampel, 8 fitur)

```
dt = DecisionTreeRegressor(random_state=42).fit(X_tr, y_tr)
rf = RandomForestRegressor(100, random_state=42,
                           n_jobs=-1).fit(X_tr, y_tr)

print(dt.score(X_tr, y_tr), dt.score(X_te, y_te))
print(rf.score(X_tr, y_tr), rf.score(X_te, y_te))
# DT  train 1.000 | test 0.623
# RF  train 0.974 | test 0.805
```

0,623

Test R^2 pohon tunggal (train = 1,000)

0,805

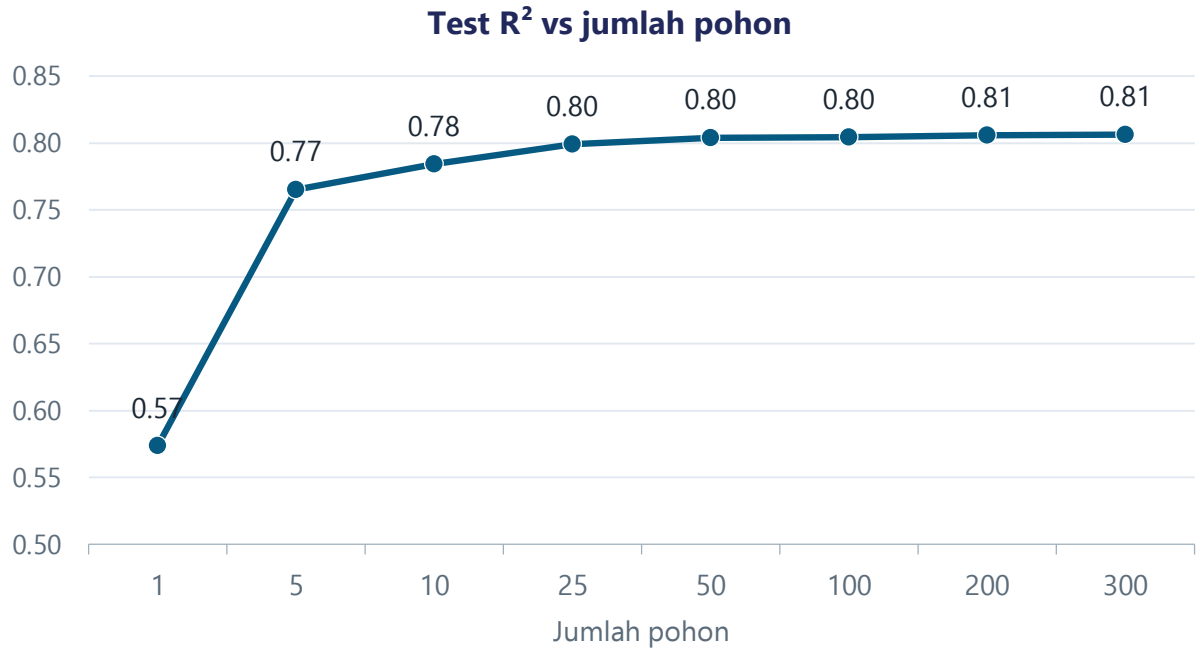
Test R^2 Random Forest (100 pohon)

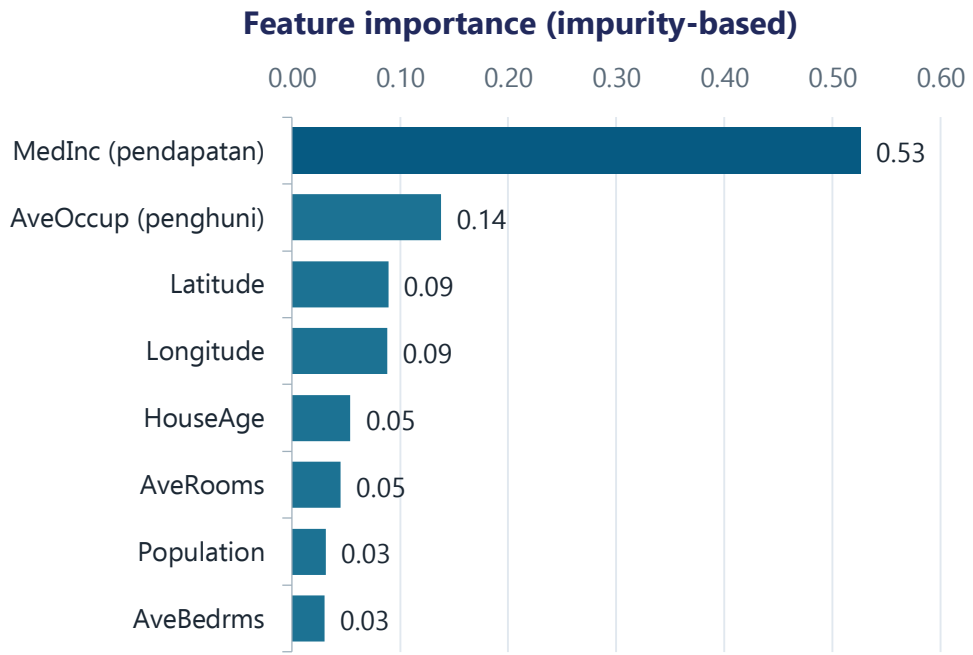
Diagnosis

Pohon tunggal menghafal data latih (variance tinggi). Merata-ratakan 100 pohon menaikkan test R^2 sebesar **0,18** dan menurunkan MAE dari 0,454 ke 0,328 (satuan $100.000).

Catatan ablasi: pada regresi, default `max_features=1.0` membuat RF ≈ Bagging (R^2 0,805 vs 0,804). Dengan `max_features=0,33` skor naik menjadi **0,810**, sedangkan 0,2 justru turun ke 0,780 → subset fitur perlu **disetel**, bukan asal kecil.

Berapa Pohon Cukup? Fitur Mana Penting?





Pelajaran: pohon makin banyak tidak menyebabkan overfit; hasil jenuh setelah ≈50–100 pohon (biaya komputasi saja yang naik).

Hati-hati: importance bawaan bias ke fitur dengan banyak nilai unik; verifikasi dengan *permutation importance*.

c **BOOSTING**

Boosting: Belajar dari Kesalahan Sebelumnya

Model dilatih berurutan; setiap model baru memfokuskan diri pada hal yang masih salah.

Analogi: Latihan Soal Berulang

Siswa mengerjakan satu paket soal, lalu **mengulang soal yang salah** pada ronde berikutnya, dan seterusnya. Setiap ronde, soal yang sulit mendapat perhatian lebih besar.



ronde 1 → temukan kesalahan → ronde 2 memperbaiki → ronde 3 …

Aspek	Bagging / RF	Boosting
Pelatihan	Paralel	Sekuensial
Base learner	Pohon dalam	Pohon dangkal (stump)
Menurunkan	Variance	Bias
Risiko overfit	Rendah	Lebih tinggi
Tuning	Mudah	Lebih sensitif
Outlier / noise	Cukup tahan	Mudah terpengaruh

c **BOOSTING**

AdaBoost: Menaikkan Bobot Sampel yang Salah

Contoh hitung: 5 sampel, bobot awal sama (0,2). Weak learner salah pada 1 sampel.

Sampel	Bobot awal	Prediksi	Bobot × e^(±α)	Bobot baru (normalisasi)
1	0,200	benar	$0{,}2 \times e^{-0.693} = 0{,}100$	0,125
2	0,200	benar	0,100	0,125
3	0,200	**SALAH**	$0{,}2 \times e^{+0.693} = 0{,}400$	**0,500**
4	0,200	benar	0,100	0,125
5	0,200	benar	0,100	0,125

- Mulai: semua sampel berbobot sama.
- Latih weak learner pada data berbobot, hitung ε.
- Hitung α (makin akurat → suara makin besar).
- Naikkan bobot sampel yang salah, turunkan yang benar.
- Ulangi; prediksi akhir = voting berbobot α.

Rumus kunci

ε = jumlah bobot sampel yang salah = 0,2

$\alpha = \frac{1}{2} \ln((1-\varepsilon)/\varepsilon) = \frac{1}{2} \ln 4 \approx 0{,}693$

$w_i \leftarrow w_i \cdot e^{\pm\alpha}$, lalu dinormalisasi

$H(x) = \text{sign}(\Sigma\ \alpha_t\ h_t(x))$

Hasil: sampel yang salah kini memegang **50%** total bobot, sehingga learner berikutnya "terpaksa" memperhatikannya.

Gradient Boosting: Model Baru Menebak Residual

Untuk loss MSE, −gradien = residual = y − F(x). Tiap pohon dangkal belajar memprediksi sisa kesalahan.

Data (y)	2	4	8	10
F_0 = rata-rata y	6	6	6	6
Residual $r_1 = y - F_0$	−4	−2	+2	+4
Pohon h_1 (stump)	−3	−3	+3	+3
$F_1 = F_0 + 0{,}5 \cdot h_1$	4,5	4,5	7,5	7,5
Residual baru r_2	−2,5	−0,5	+0,5	+2,5

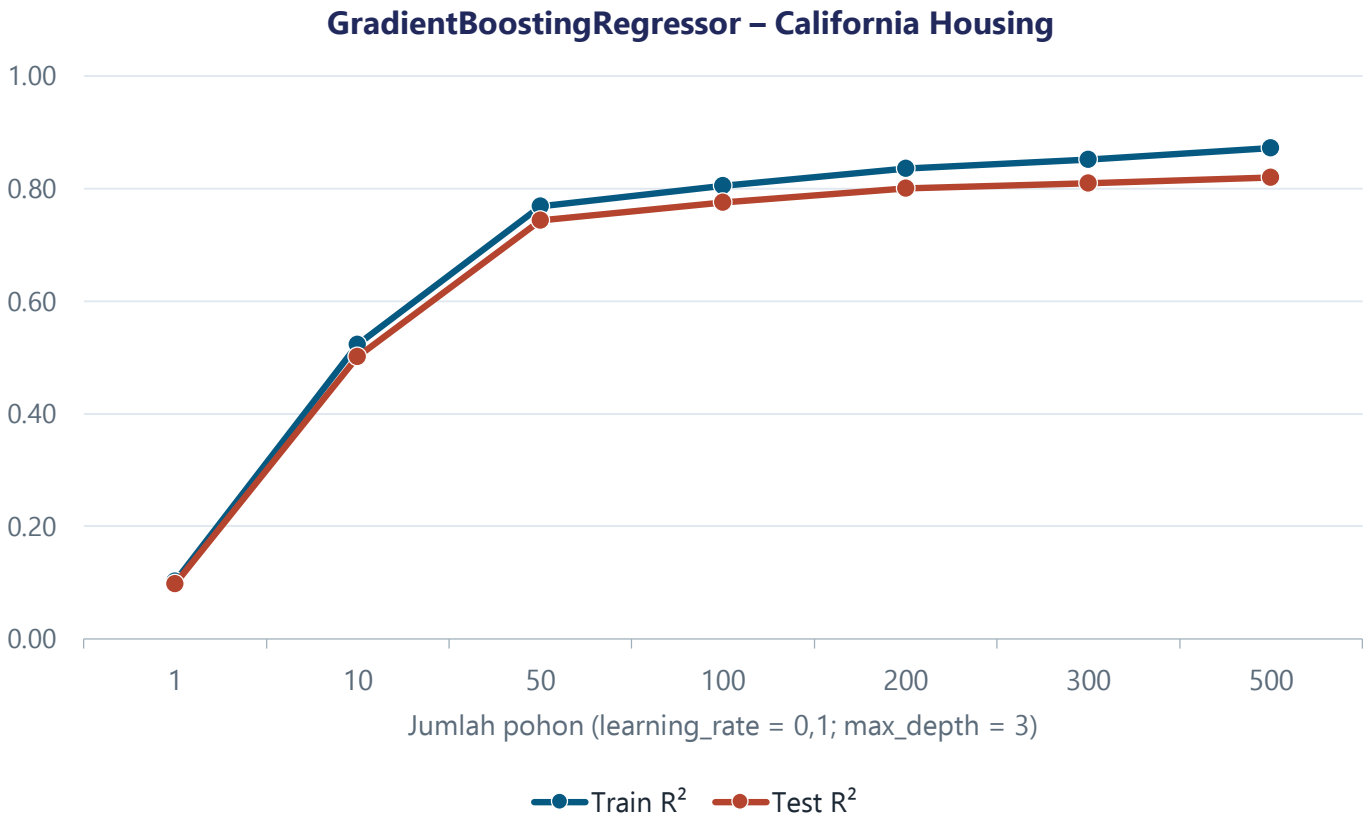
Residual mengecil: kuadrat error turun dari (16+4+4+16) = 40 menjadi (6,25+0,25+0,25+6,25) = 13.

Algoritma

1. $F_0(x)$ = rata-rata y
2. Untuk m = 1…M: hitung residual $r = y - F_{m-1}(x)$
3. Latih pohon dangkal h_m pada r
4. $F_m = F_{m-1} + \eta \cdot h_m(x)$

Hubungan dengan Pertemuan 3: Gradient Descent memperbarui **parameter** θ; Gradient Boosting memperbarui **fungsi** F(x) searah −gradien loss. η (learning rate/shrinkage) berperan sama seperti langkah kaki di gunung berkabut.

Boosting Butuh Dihentikan Tepat Waktu



0,872
Train R^2 pada 500 pohon

0,820
Test R^2 pada 500 pohon

Diagnosis
Test R^2 terus naik, tetapi **celah train–test melebar** (0,031 → 0,052) seiring ronde bertambah.

```
GradientBoostingRegressor(
    n_estimators=2000, learning_rate=0.05,
    n_iter_no_change=20,
    validation_fraction=0.1)
```

c BOOSTING

Boosting Modern: XGBoost, LightGBM, CatBoost

Gradient Boosting yang direkayasa untuk akurasi, kecepatan, dan ketahanan pada data tabular.

Fitur	Manfaat
Regularisasi L1/L2 pada bobot daun	Sama prinsipnya dengan Ridge/Lasso (Pertemuan 3): menahan overfitting
Shrinkage (η), subsample, colsample	Menurunkan korelasi antar pohon, mirip semangat Random Forest
Split berbasis histogram	Jauh lebih cepat pada data besar (LightGBM, HistGradientBoosting)
Penanganan missing value bawaan	Model belajar arah terbaik untuk nilai kosong
Early stopping	Berhenti saat skor validasi tidak membaik
Pengaturan imbalance	Parameter seperti scale_pos_weight (dibahas di bagian 3)

XGBoost

Regularisasi eksplisit, sangat populer di kompetisi.

LightGBM

Pertumbuhan pohon leaf-wise, cepat dan hemat memori.

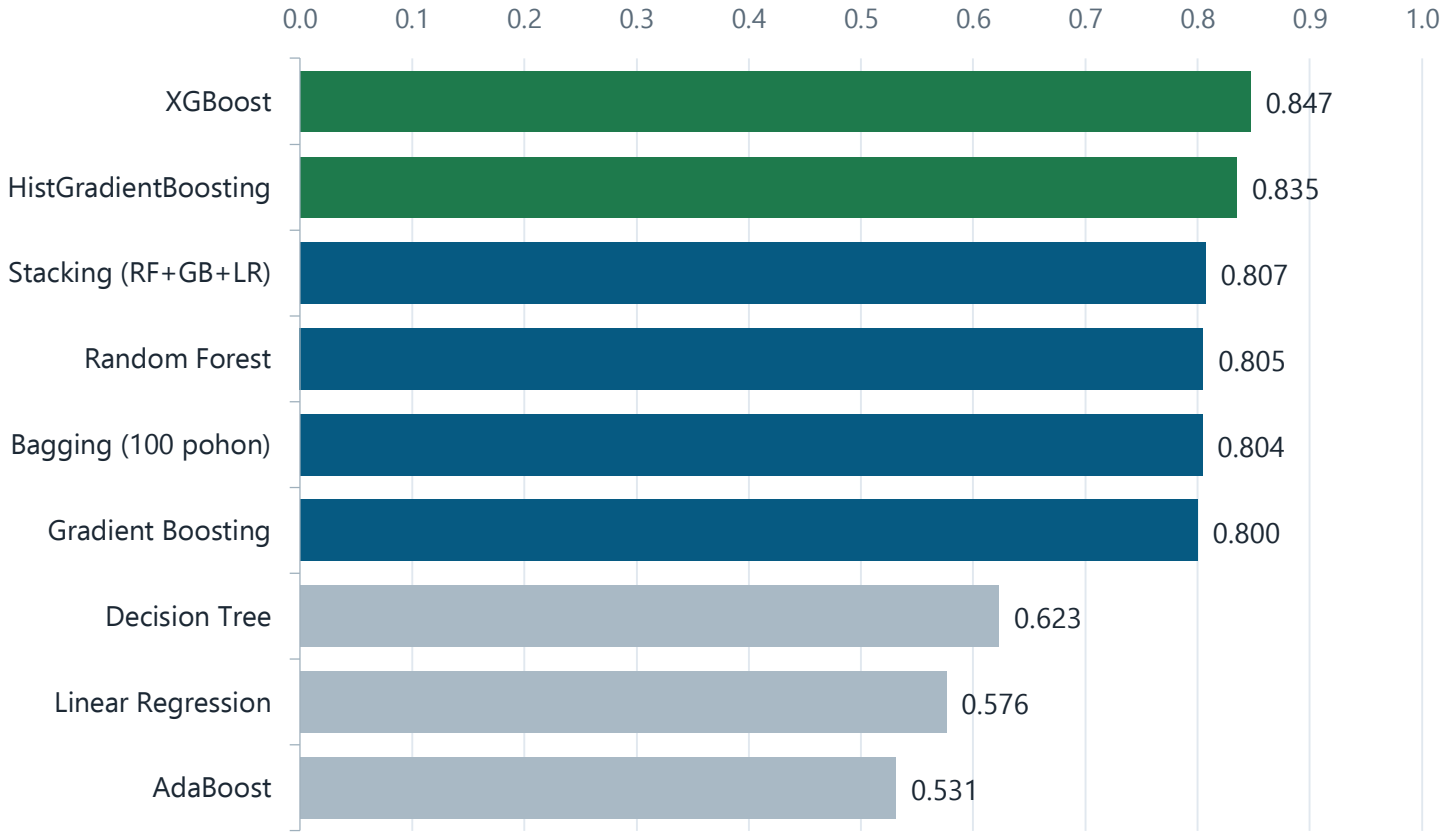
CatBoost

Penanganan fitur kategorikal bawaan.

HistGradientBoosting

Versi scikit-learn, histogram-based, tanpa dependensi tambahan.

Adu Semua Model pada Data yang Sama



Ensemble menang telak

Dari R^2 0,576 (linier) dan 0,623 (pohon tunggal) menjadi 0,80–0,85.

Stacking mahal, untung tipis

R^2 0,807 vs RF 0,805, tetapi 39 detik vs 5 detik latih.

AdaBoost default lemah di regresi

R^2 0,531; AdaBoost lebih cocok untuk klasifikasi. Tidak ada model yang selalu terbaik.

Voting: Hard vs Soft

Menggabungkan model **berbeda jenis** (mis. Logistic Regression, SVM, Random Forest).

Model	P(kelas 1)	Hard vote
A Logistic Regression	0,45	kelas 0
B SVM	0,45	kelas 0
C Random Forest	0,95	kelas 1
Hasil	**rata-rata 0,617**	**2 vs 1 → kelas 0**

Soft voting menghasilkan kelas 1 (0,617 > 0,5) karena model C sangat yakin. Hard voting hanya menghitung suara, mengabaikan tingkat keyakinan.

Soft voting mensyaratkan model yang menghasilkan probabilitas **terkalibrasi** (predict_proba).

Kode: VotingClassifier

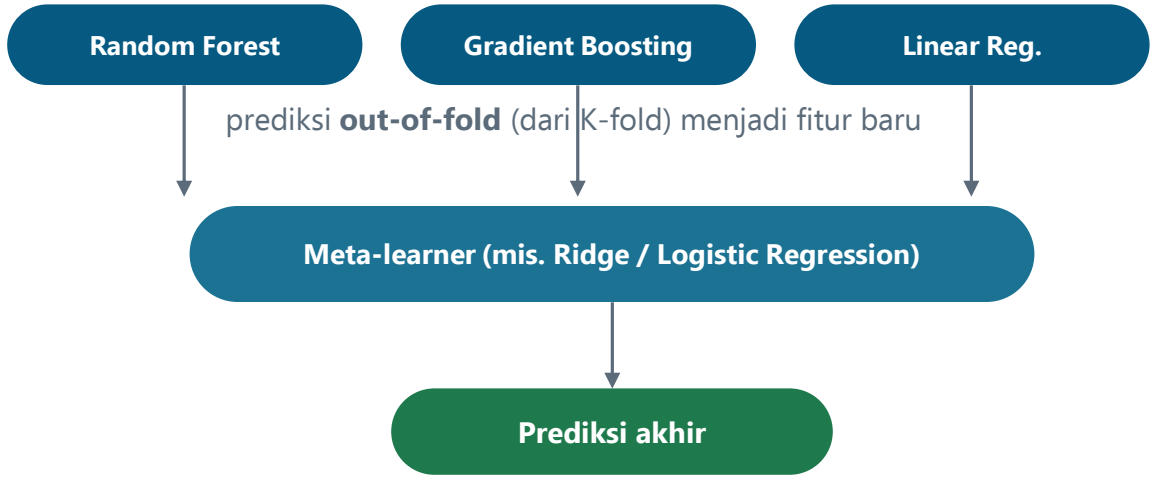
```
from sklearn.ensemble import VotingClassifier

vote = VotingClassifier(
  estimators=[
    ('lr',  LogisticRegression()),
    ('svm', SVC(probability=True)),
    ('rf',  RandomForestClassifier())],
  voting='soft')
vote.fit(X_train, y_train)
```

Ganti `voting='hard'` untuk suara mayoritas biasa.

Stacking: Model yang Belajar dari Model

Prediksi model dasar (level-0) menjadi fitur bagi meta-learner (level-1).



Awas leakage: jika meta-learner dilatih dari prediksi **in-sample** model dasar, ia belajar dari prediksi yang terlalu bagus. Gunakan parameter `cv=5`.

0,807

Test R^2 Stacking (RF+GB+LR)

0,805

Test R^2 Random Forest saja

Kode

```
StackingRegressor(
  estimators=[('rf', RF), ('gb', GB), ('lr', LR)],
  final_estimator=LinearRegression(),
  cv=5)
```

Waktu latih **39 detik** vs 5 detik (RF) → stacking baru layak bila ada keragaman model dan margin akurasi sangat berarti.

d VOTING & STACKING

Kapan Memakai yang Mana?

Ringkasan perbandingan metode ensemble.

	Bagging / Random Forest	Boosting	Stacking / Voting
Tujuan utama	Menurunkan variance	Menurunkan bias	Memadukan kekuatan model berbeda
Pelatihan	Paralel, mudah	Sekuensial	Berlapis, mahal
Risiko overfit	Rendah	Sedang–tinggi (perlu early stopping)	Sedang (hati-hati leakage)
Tuning	Sedikit hyperparameter	Sensitif (η, depth, jumlah pohon)	Banyak komponen
Cocok untuk	Baseline kuat & cepat; data noisy	Akurasi maksimum data tabular	Tahap akhir kompetisi / margin tipis

Aturan praktis: mulai dari **Random Forest** (baseline kuat, sedikit tuning) → coba **Gradient Boosting** untuk akurasi lebih tinggi → baru pertimbangkan stacking.

UNIVERSITAS
MIKROSKIL

Validation Strategy

Hold-out • Data Leakage • Cross-Validation • Hyperparameter Tuning

e **STRATEGI VALIDASI**

Tiga Peran Data: Train, Validation, Test

Tujuan validasi: memperkirakan performa pada data baru yang belum pernah dilihat (generalization error).



Train — Latihan soal

Model belajar parameter (bobot, split pohon).

Validation — Try-out

Memilih model dan hyperparameter; boleh dipakai berkali-kali.

Test — Ujian akhir

Estimasi performa final. Dipakai **sekali saja**, di akhir.

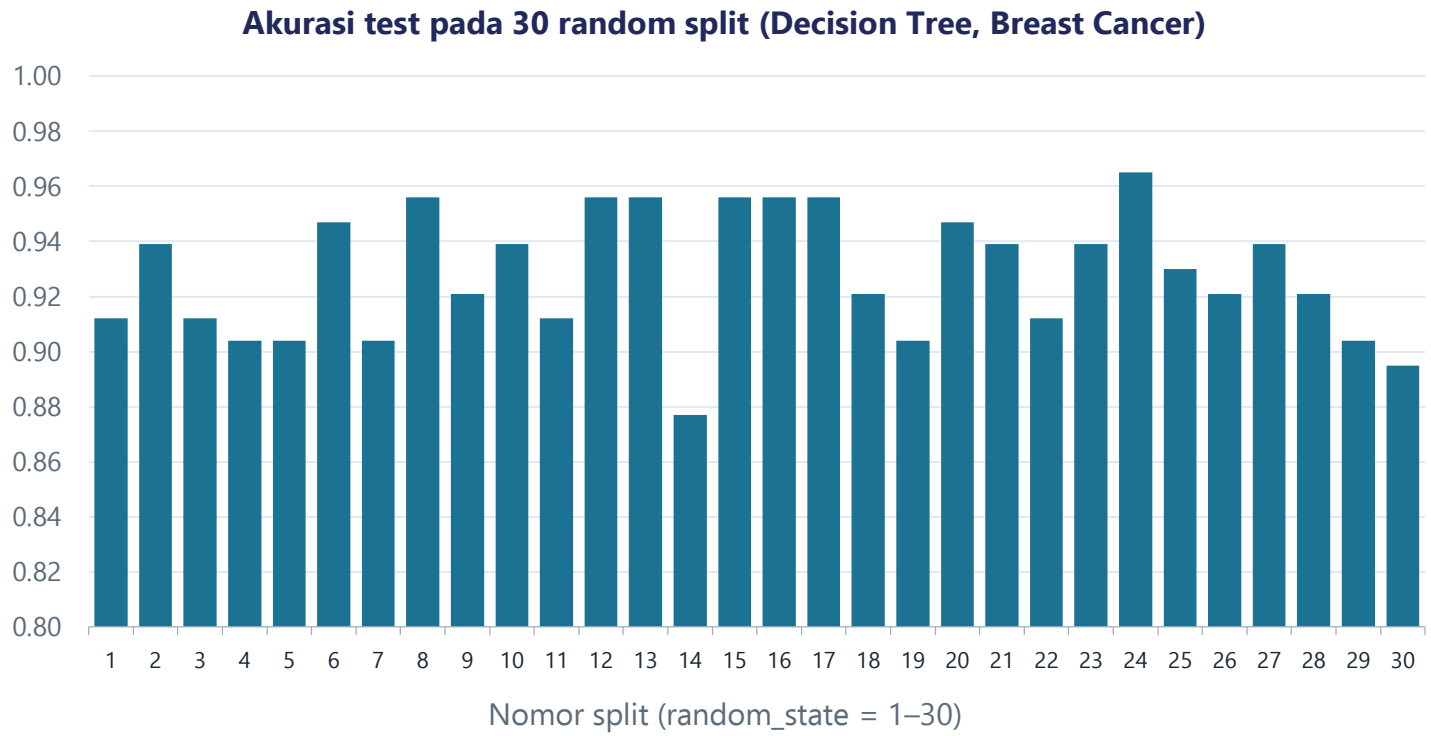
Aturan emas: jika test set dipakai untuk memilih model atau menyetel hyperparameter, ia berubah menjadi validation set dan angka akhirnya menjadi **terlalu optimis**. Seperti siswa yang berlatih memakai soal ujian akhir.

e

STUDI KASUS NYATA

STRATEGI VALIDASI

Satu Kali Split Bisa Menipu



0,877

akurasi terendah

0,965

akurasi tertinggi

Diagnosis

Model dan data sama, hanya pembagian acak yang berbeda: rata-rata 0,928, simpangan baku 0,023. Test set 114 sampel → satu kesalahan ≈ 0,9 poin.

Pelajaran: laporkan **rata-rata ± simpangan baku** dari banyak split (cross-validation), bukan satu angka.

e **STRATEGI VALIDASI**

Data Leakage: Informasi yang "Bocor" ke Training

Model memperoleh informasi yang tak tersedia saat dipakai nyata → skor tampak hebat, performa asli buruk.

1 Preprocessing sebelum split

Scaler, imputer, seleksi fitur, atau SMOTE di-fit pada seluruh data.

→ Bungkus dalam **Pipeline**; fit hanya pada fold latih.

2 Target leakage

Fitur baru diketahui setelah target terjadi, mis. status_refund untuk memprediksi churn.

→ Tanyakan: apakah fitur ini ada saat prediksi dibuat?

3 Duplikat / Group leakage

Pasien/rumah/pengguna yang sama muncul di train dan test.

→ Pisahkan per grup: **GroupKFold**.

4 Temporal leakage

Split acak pada data waktu: masa depan dipakai untuk memprediksi masa lalu.

→ Gunakan split berurutan: **TimeSeriesSplit**.

e

STUDI KASUS NYATA

STRATEGI VALIDASI

Leakage pada Data Murni Acak

Notebook Pertemuan 4 – 100 sampel × 5.000 fitur noise

```
X = rng.randn(100, 5000)         # 5000 fitur NOISE
y = rng.randint(0, 2, 100)        # label ACAK

# SALAH: seleksi fitur memakai SEMUA data
Xs = SelectKBest(f_classif, k=20).fit_transform(X, y)
cross_val_score(LogisticRegression(), Xs, y, cv=5)

# BENAR: seleksi di dalam tiap fold
pipe = Pipeline([('sel', SelectKBest(f_classif, k=20)),
                 ('lr', LogisticRegression())])
cross_val_score(pipe, X, y, cv=5)
```

0,83

akurasi CV — seleksi fitur di luar fold

0,51

akurasi CV — di dalam Pipeline

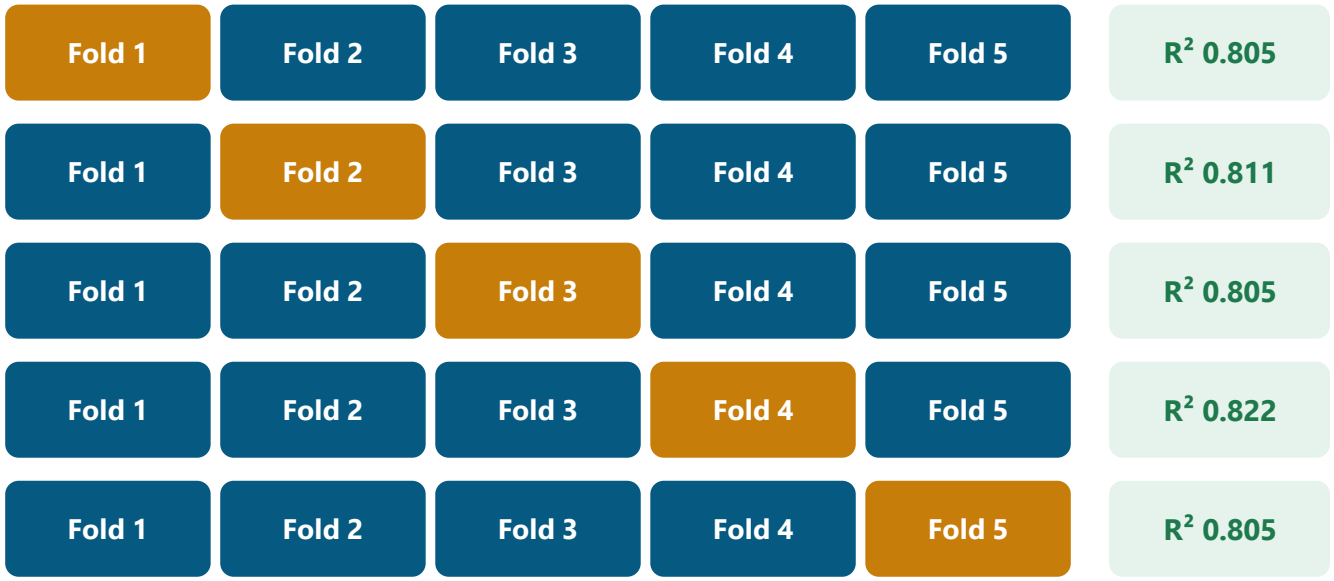
Diagnosis

Labelnya acak, jadi akurasi seharusnya ≈ 50%. Angka 83% muncul karena seleksi fitur sudah "mengintip" label seluruh data.

Aturan emas: apa pun yang memanggil `.fit()` (scaler, imputer, seleksi fitur, resampling, model) harus hanya melihat data **latih** pada fold tersebut. Cara termudah: `Pipeline`.

K-Fold Cross-Validation

Bagi data latih menjadi K bagian; tiap bagian bergantian menjadi validation, sisanya untuk latihan.



Amber = validation fold, biru = fold latih

$$CV = (1/K)\ \Sigma\ skor_k$$

laporkan juga simpangan baku

0,810 ± 0,006

Random Forest, 5-fold, California Housing

Manfaat: seluruh data latih dipakai untuk validasi secara bergantian dan kita mendapat **estimasi + ketidakpastiannya**. Harga: biaya latih × K.

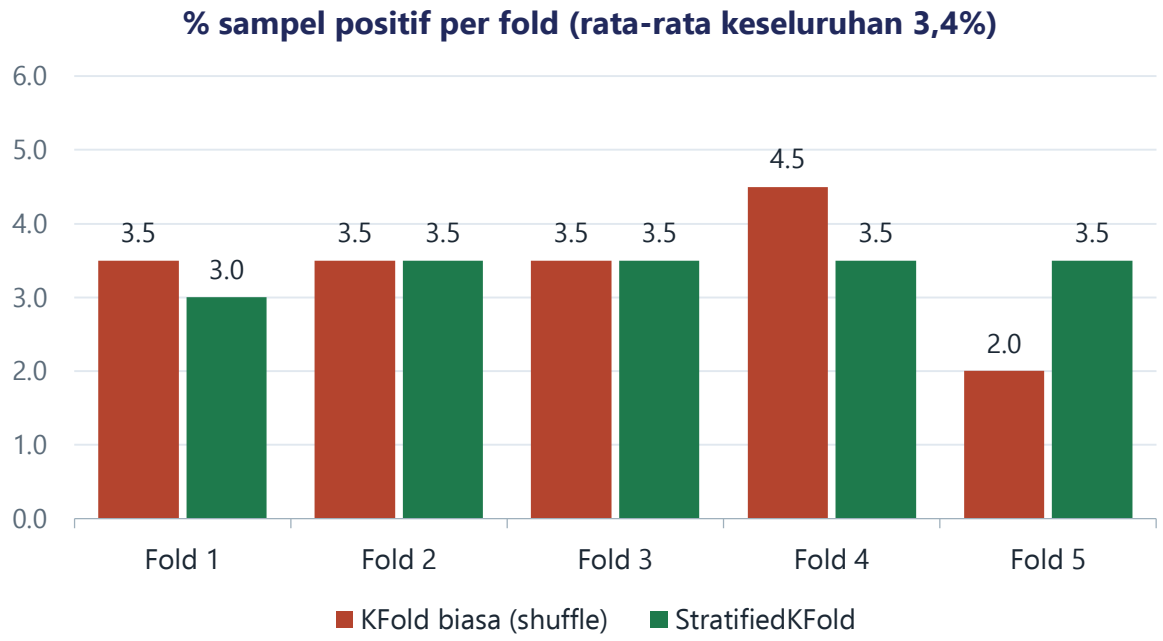
Memilih Varian CV yang Sesuai Data

Pilihan skema CV harus mencerminkan cara model akan dipakai di dunia nyata.

Varian (scikit-learn)	Cara kerja	Kapan dipakai
KFold	K bagian acak	Data i.i.d., regresi
StratifiedKFold	Proporsi kelas sama di tiap fold	Klasifikasi, **terutama imbalanced**
RepeatedStratifiedKFold	Ulangi K-fold dengan acakan berbeda	Data kecil / positif sedikit → estimasi stabil
GroupKFold	Satu grup tidak terpisah antar fold	Banyak sampel per pasien/pengguna/rumah
TimeSeriesSplit	Latih masa lalu, uji masa depan	Data deret waktu
LeaveOneOut	K = n	Data sangat kecil; mahal dan varians tinggi

Pemilihan K: K kecil → bias estimasi lebih besar (latih dengan data lebih sedikit); K besar → mahal dan variansnya lebih tinggi. K = 5 atau 10 adalah kompromi umum.

Stratified vs Biasa, dan Time Series Split



1.000 sampel dengan 3,4% positif. KFold biasa: fold 5 hanya 2,0% dan fold 4 sebesar 4,5%. StratifiedKFold menjaga 3,0–3,5%.



Latih selalu **sebelum** uji. Data masa depan (abu-abu) tidak pernah dipakai untuk memprediksi masa lalu.

Hyperparameter Tuning: Mencari Pengaturan Terbaik

Parameter dipelajari dari data; hyperparameter ditetapkan **sebelum** pelatihan dan dipilih lewat validasi.

Parameter
Dipelajari dari data: bobot θ regresi, nilai daun pohon.
Hyperparameter
Diatur manusia: max_depth, learning_rate, α (Ridge/Lasso), n_estimators.

Kode: GridSearchCV

```
param_grid = {
  'learning_rate': [0.03, 0.1, 0.3],
  'max_depth': [3, 6, None],
  'max_iter': [100, 300, 600]}
gs = GridSearchCV(model, param_grid,
                  cv=5, scoring='r2')
gs.fit(X_train, y_train)
```

Strategi	Cara kerja	Catatan
Grid Search	Coba semua kombinasi	Menyeluruh, tetapi biaya meledak
Random Search	Sampel kombinasi acak	Sering hampir sama bagusnya, jauh lebih murah
Successive Halving	Buang kandidat buruk lebih awal	Efisien untuk ruang pencarian besar
Bayesian (mis. Optuna)	Belajar dari percobaan sebelumnya	Efisien bila satu percobaan mahal

Ingat: tuning dilakukan **hanya pada data latih** lewat CV. Test set tetap tidak tersentuh sampai evaluasi akhir.

STUDI KASUS NYATA

CROSS-VALIDATION & TUNING

Grid vs Random Search pada Data yang Sama

Metode	Fit total	Waktu	CV R^2	Test R^2
Default (tanpa tuning)	1	—	—	0,835
GridSearchCV (54 kombinasi × 5 fold)	270	58 detik	0,8406	0,8462
RandomizedSearchCV (10 iterasi × 5 fold)	50	12 detik	0,8400	0,8456

+0,011

Test R^2 setelah tuning (0,835 → 0,846)

÷ 5

biaya Random Search dibanding Grid

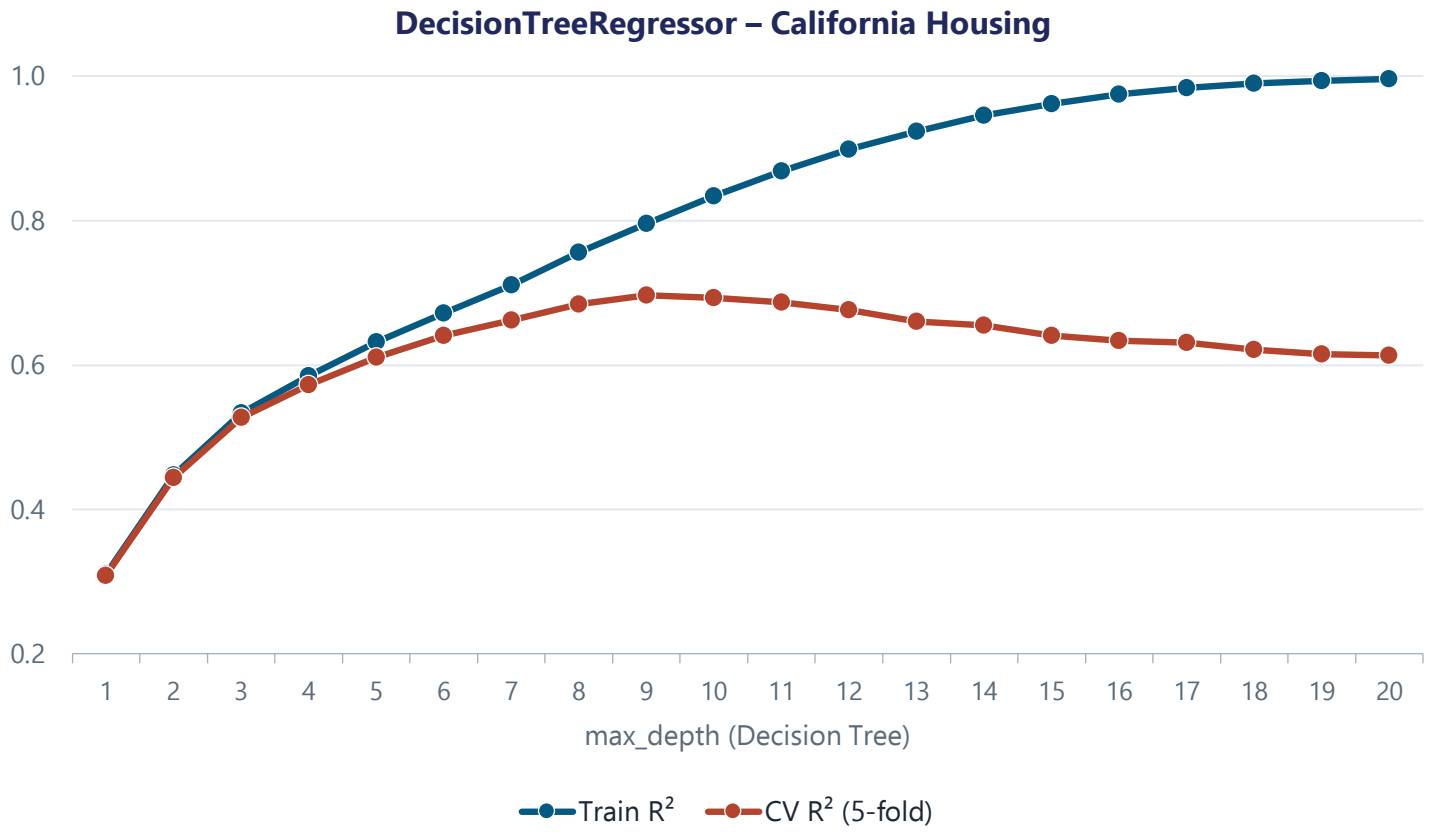
Pelajaran: tuning memberi perbaikan nyata tetapi kecil; Random Search hampir menyamai Grid dengan seperlima biaya.

HistGradientBoostingRegressor – California Housing

```
best = {'learning_rate': 0.03, 'max_iter': 600,
        'max_depth': None, 'min_samples_leaf': 50}

rs = RandomizedSearchCV(
    HistGradientBoostingRegressor(),
    param_grid, n_iter=10, cv=5, random_state=42)
```

Membaca Validation Curve



max_depth 1–5 — Underfitting

Train dan CV sama-sama rendah, celah kecil → model terlalu sederhana (high bias).

max_depth ≈ 9 — Titik terbaik

CV R^2 tertinggi 0,697 (train 0,796).

max_depth 20 — Overfitting

Train 0,996 tetapi CV turun ke 0,613 → celah lebar (high variance).

f CROSS-VALIDATION & TUNING

Protokol Validasi yang Benar

Urutan kerja yang menjaga angka akhir tetap jujur.

1. Pisahkan test	2. CV pada train	3. Fit ulang	4. Uji sekali	5. Produksi
Sisihkan 20% data. Kunci.	Bandingkan model dan tuning hyperparameter.	Latih model terbaik pada seluruh data train.	Hitung skor pada test set. Satu kali.	Opsional: latih ulang dengan semua data.

Dosa besar validasi

- Menyetel model berdasarkan skor test set.
- Preprocessing/resampling sebelum split.
- Random split pada data waktu.
- Melaporkan hanya skor terbaik dari banyak percobaan.

Nested Cross-Validation

Untuk data kecil: CV **luar** menaksir performa, CV **dalam** (di tiap fold luar) menyetel hyperparameter. Estimasinya lebih jujur, biayanya jauh lebih besar.

UNIVERSITAS
MIKROSKIL

Imbalanced Data

Masalah • Metrik yang Tepat • Teknik Penanganan • Pipeline yang Benar

Apa Itu Data Tidak Seimbang?

Satu kelas (mayoritas) jauh lebih banyak daripada kelas lain (minoritas), padahal minoritas biasanya yang paling penting.

284.807
transaksi kartu kredit (dataset fraud ULB)

492
transaksi fraud

0,172%
proporsi fraud

1 : 578
imbalance ratio (minoritas : mayoritas)

Domain	Kelas minoritas	Biaya salah
Keuangan	Transaksi fraud	Fraud lolos = kerugian langsung
Kesehatan	Pasien penyakit langka	Pasien sakit dinyatakan sehat
Manufaktur	Produk cacat	Cacat lolos ke pelanggan
Keamanan siber	Serangan / intrusi	Serangan tidak terdeteksi

Kunci: biaya **False Negative** (kasus penting terlewat) umumnya jauh lebih mahal daripada **False Positive**. Model dan metrik harus mencerminkan hal itu.

Accuracy Paradox: 98,9% yang Tidak Berguna

Data sintetis: 20.000 sampel, 1,1% positif. Test set 5.000 sampel berisi **55 positif**. Model "bodoh": **selalu menebak negatif**.

	Prediksi +	Prediksi –
Aktual +	TP 0	FN 55
Aktual –	FP 0	TN 4945

98,9%
accuracy

0%
recall kasus positif

Model	Accuracy	Recall
Selalu negatif (Dummy)	98,9%	0,0%
Logistic Regression	99,1%	16,4%
Logistic Reg. class_weight	78,7%	78,2%

Kesimpulan: accuracy tidak boleh menjadi satu-satunya metrik pada data imbalanced.

Pelajaran: model terbaik menurut accuracy (99,1%) hanya menangkap 16% kasus. Model yang menangkap 78% kasus justru "terlihat lebih buruk" menurut accuracy.

Confusion Matrix dan Metrik Turunannya

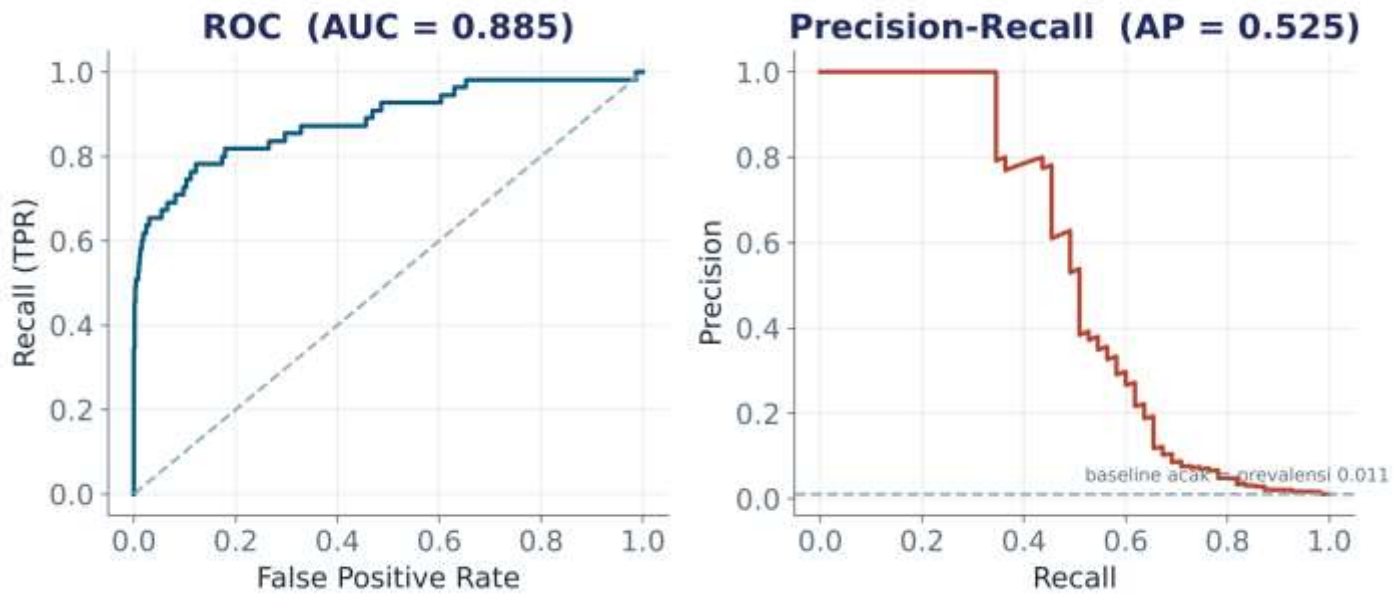
Analogi: alarm kebakaran. Kelas positif = ada kebakaran.

	Prediksi +	Prediksi −
Aktual +	TP terdeteksi	FN terlewat
Aktual −	FP alarm palsu	TN aman

TP kebakaran terdeteksi • **FN** kebakaran terlewat
FP alarm palsu • **TN** aman, tidak berbunyi

Recall (Sensitivity)	**TP / (TP + FN)** Dari semua kebakaran, berapa yang terdeteksi?
Precision	**TP / (TP + FP)** Dari semua alarm yang berbunyi, berapa yang benar?
F1-score	**2PR / (P + R)** Rata-rata harmonik; rendah jika salah satu rendah.
Balanced accuracy	**(Recall + Specificity) / 2** Accuracy yang adil untuk tiap kelas.

ROC-AUC vs PR-AUC



0,885
ROC-AUC (tampak bagus)

0,525
Average Precision (PR-AUC)

Mengapa berbeda?
FPR dibagi jumlah negatif yang sangat besar, sehingga ribuan alarm palsu tetap tampak kecil. PR-AUC langsung memuat precision.

Pedoman: data imbalanced dan fokus pada kelas positif → laporkan **PR-AUC (average precision)**, Precision, Recall, F1. Baseline acak PR = proporsi positif (0,011).

Ingat: ROC-AUC tetap berguna untuk menilai kemampuan **ranking**.

Peta Strategi Penanganan

Mulai dari yang paling sederhana. Resampling bukan langkah pertama.

1. Evaluasi

Ukur dengan benar

- Metrik: PR-AUC, F1, Recall
- StratifiedKFold
- Baseline Dummy

2. Algoritma

Ubah fungsi loss

- class_weight='balanced'
- scale_pos_weight (XGBoost)
- BalancedRandomForest

3. Keputusan

Geser ambang

- Threshold moving
- Dipilih di validation set
- Berdasar biaya FP vs FN

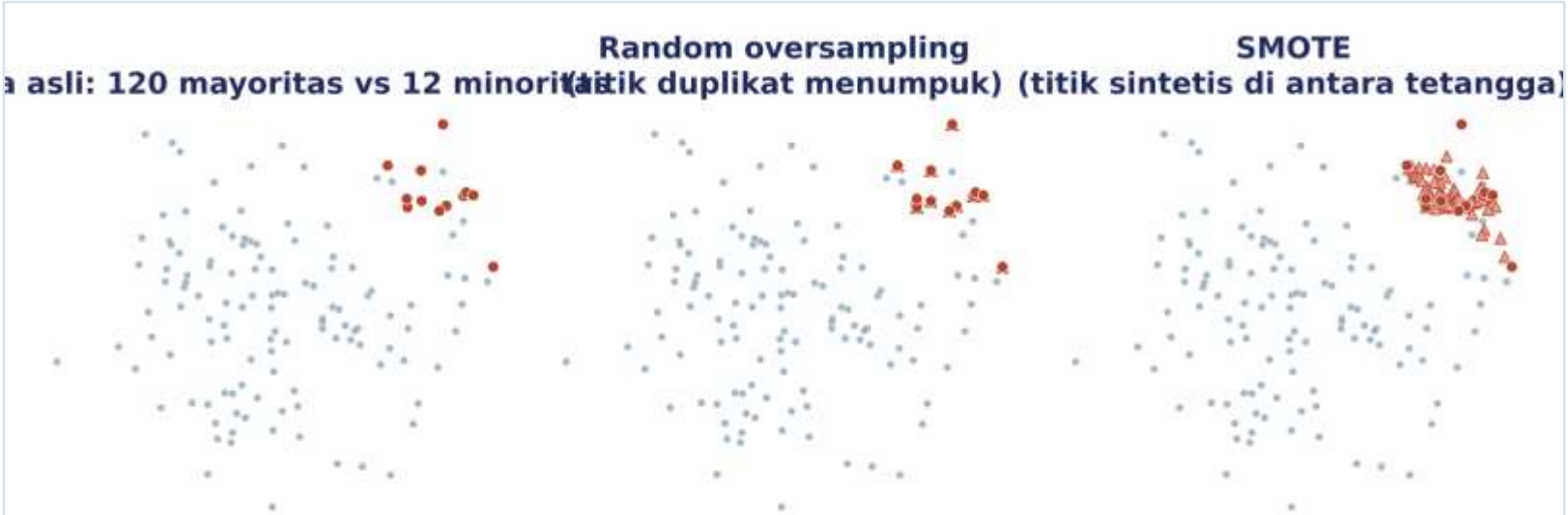
4. Data

Ubah komposisi data

- Random under/over-sampling
- SMOTE & variannya
- Hanya di dalam Pipeline

Urutan kerja: evaluasi yang benar → class_weight / threshold → baru resampling bila masih perlu.

Resampling: Random vs SMOTE



Rumus SMOTE

$x_{baru} = x_i + \lambda (x_{tetangga} - x_i)$

λ acak di [0, 1]. Titik sintetis diletakkan pada garis antara sampel minoritas dan salah satu k tetangga terdekatnya.

Random undersampling

Buang sebagian mayoritas. Cepat, tetapi **membuang informasi**.

Random oversampling

Duplikasi minoritas. Sederhana, tetapi mudah **overfit** (titik kembar).

SMOTE

Buat titik sintetis. Hati-hati **noise/overlap**; SMOTE-NC untuk fitur kategorikal.

Class Weight: Hukuman Berbeda untuk Tiap Kelas

Tidak mengubah data; mengubah seberapa mahal kesalahan pada kelas minoritas.

$$w_k = n / (K \cdot n_k)$$

n = total sampel, K = jumlah kelas, n_k = sampel kelas k

Weighted log-loss: $L = -\Sigma\, w_{y_i} [\, y_i \log p_i + (1-y_i) \log(1-p_i) \,]$

45,7

bobot kelas positif (164 dari 15.000)

0,51

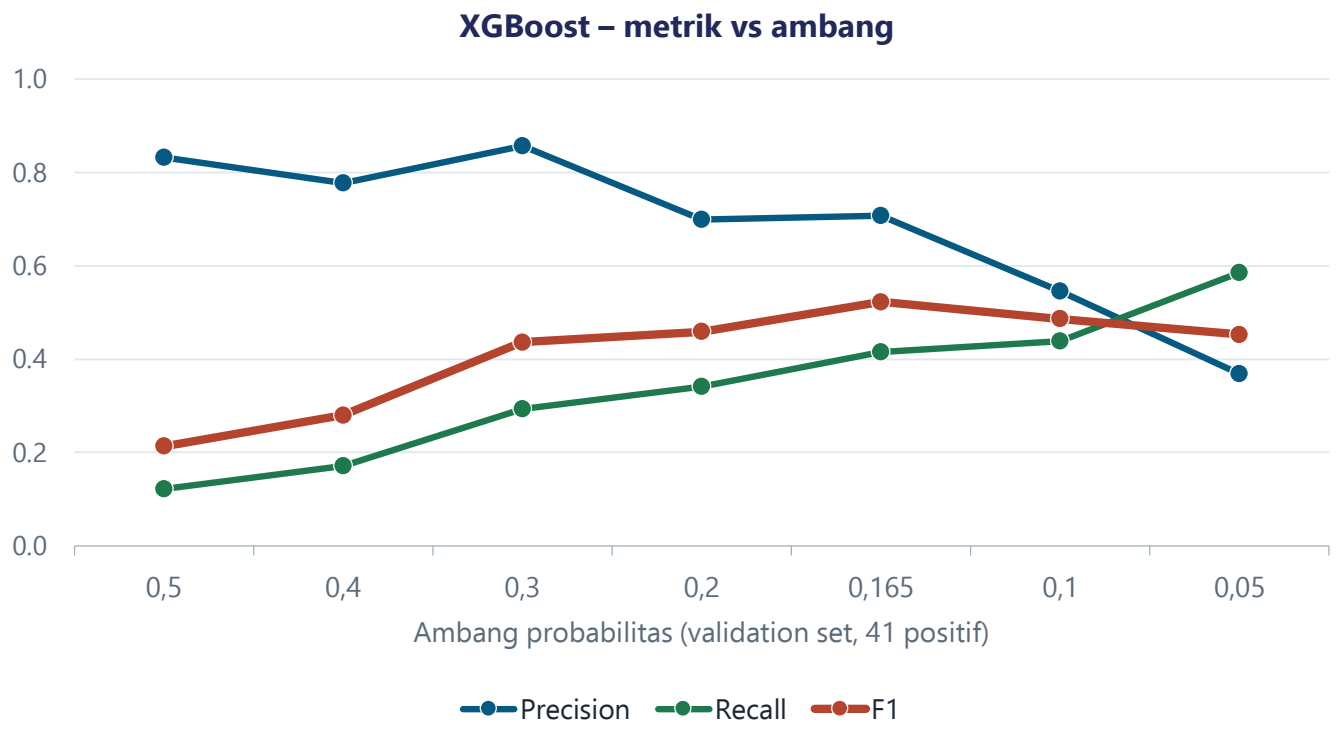
bobot kelas negatif (14.836)

Kode

```
LogisticRegression(class_weight='balanced')
RandomForestClassifier(
    class_weight='balanced_subsample')
XGBClassifier(
    scale_pos_weight=n_neg / n_pos)
```

Efek nyata: LogReg class_weight menaikkan recall 16% → 78%, tetapi precision turun 90% → 3,9%: 1.054 alarm palsu untuk 43 kasus benar. Ini menggeser **titik operasi**, bukan membuat model "lebih pintar".

Threshold Moving: Geser Ambang Keputusan



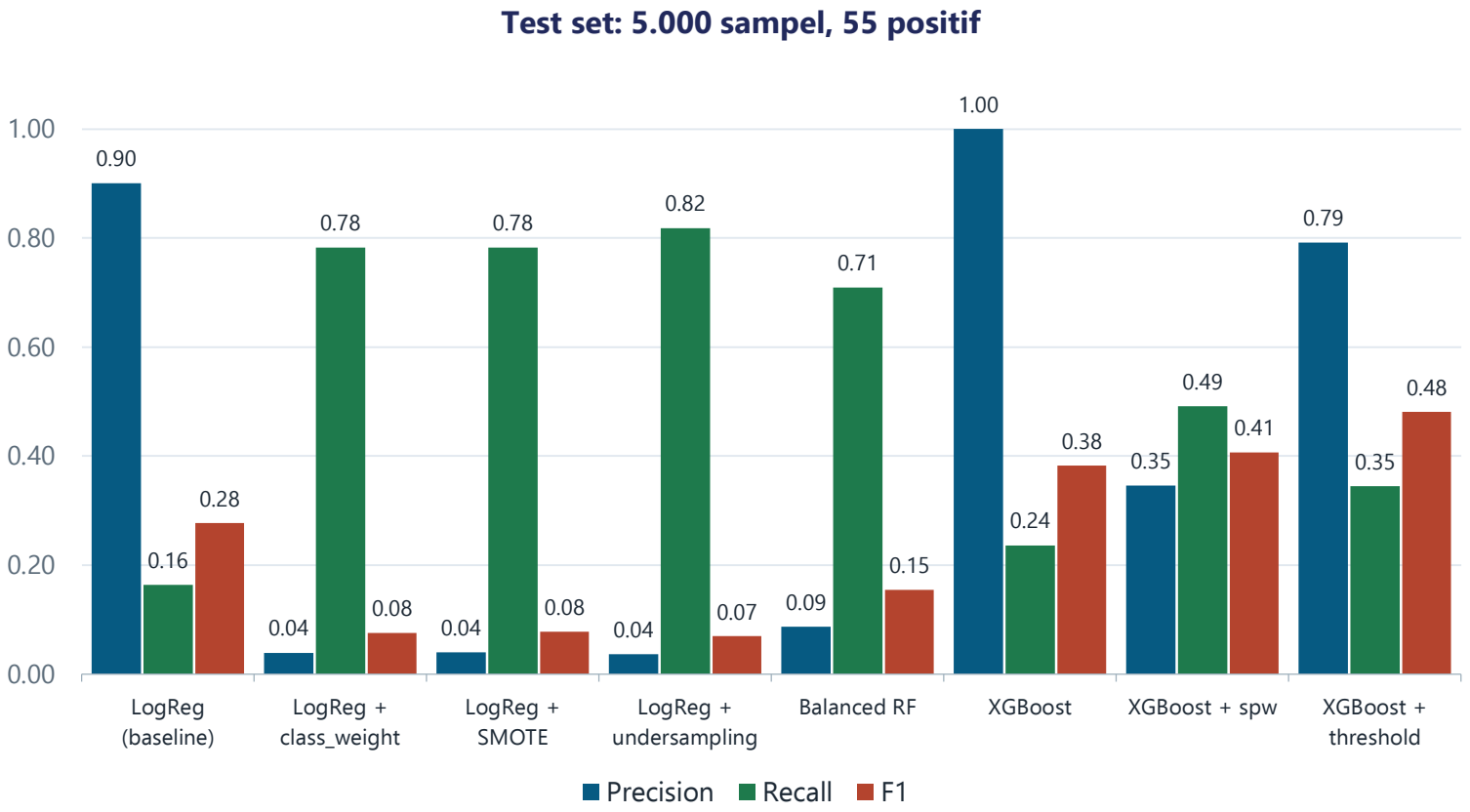
F1 validation tertinggi pada ambang 0,165 (bukan 0,5).

Hasil di test set (5.000 sampel, 55 positif)

Ambang	Precision	Recall	F1
0,5 (default)	1,000	0,218	0,358
0,165 (dipilih di validation)	0,792	0,345	0,481

Aturan: ambang dipilih di **validation set** (atau lewat CV), jangan di test set. Tanpa melatih ulang, F1 naik 0,358 → 0,481. Pemilihan ambang idealnya mengikuti **biaya bisnis** FP vs FN.

Adu Strategi pada Data yang Sama



Recall naik, precision runtuh

class_weight, SMOTE, undersampling: recall ≈ 0,78–0,82 tetapi precision ≈ 0,04 → F1 hanya ≈ 0,07.

Model kuat + ambang tepat

XGBoost + threshold mencapai F1 terbaik (0,481).

Waspada noise

Hanya 55 positif: selisih kecil antar metode belum tentu bermakna.

Harga yang Dibayar: TP vs Alarm Palsu

Hitungan nyata pada 5.000 sampel uji (55 positif).

Strategi	TP	FP	FN	Recall	Precision	F1	PR-AUC
LogReg (baseline)	9	1	46	0.16	0.90	0.28	0.37
LogReg + class_weight	43	1.054	12	0.78	0.04	0.07	0.29
LogReg + SMOTE	43	1.020	12	0.78	0.04	0.08	0.30
LogReg + undersampling	45	1.212	10	0.82	0.04	0.07	0.26
Balanced Random Forest	39	411	16	0.71	0.09	0.15	0.40
XGBoost	13	0	42	0.24	1.00	0.38	0.53
XGBoost + scale_pos_weight	27	51	28	0.49	0.35	0.41	0.40
XGBoost + threshold	19	5	36	0.35	0.79	0.48	0.49

Perhatikan: PR-AUC (kemampuan ranking) hampir tidak membaik oleh resampling/class_weight. Yang berubah adalah **titik operasi**.

Perangkap: SMOTE Sebelum CV

imbalanced-learn – data sintetis 1,1% positif

```
# SALAH: SMOTE pada seluruh data, baru CV
Xs, ys = SMOTE().fit_resample(X_train, y_train)
cross_val_score(RandomForestClassifier(), Xs, ys,
                cv=5, scoring='f1')

# BENAR: resampling di dalam Pipeline (per fold)
from imblearn.pipeline import Pipeline
pipe = Pipeline([('smote', SMOTE()),
                 ('rf', RandomForestClassifier())])
cross_val_score(pipe, X_train, y_train,
                cv=5, scoring='f1')
```

0,995

F1 CV — SMOTE di luar fold

0,256

F1 CV — di dalam Pipeline

Diagnosis

F1 pada test set asli: **0,310**. Angka 0,995 palsu: titik sintetis (dan salinannya) muncul di fold latih dan fold validasi.

Aturan: resampling hanya boleh mengubah **data latih per fold**. Validation fold dan test set harus mempertahankan distribusi asli.

Alur Kerja untuk Data Imbalanced

Ringkasan langkah yang bisa langsung dipakai pada project.

1. Stratified split	2. Baseline	3. Algoritma	4. Ambang	5. Resampling
Jaga proporsi kelas di train/test.	Dummy + model sederhana; metrik PR.	class_weight / scale_pos_weight.	Pilih di validation set.	Jika perlu, di dalam Pipeline.

Pelaporan yang jujur

- Precision, Recall, F1, PR-AUC — bukan accuracy saja.
- Sertakan confusion matrix.
- Sebutkan jumlah kasus positif di test.

Hati-hati: positif sedikit

Dengan 55 positif, satu kasus berubah ≈ 1,8 poin recall. Gunakan **RepeatedStratifiedKFold** dan laporkan rata-rata ± simpangan baku.

? CEK PEMAHAMAN

Pertanyaan Diskusi

Jawab singkat, lalu diskusikan dengan teman sebangku (3 menit).

1. Mengapa Random Forest menurunkan variance tetapi hampir tidak mengubah bias? Kapan boosting lebih tepat?

2. Sebuah tim menyetel hyperparameter berdasarkan skor test set lalu melaporkan skor yang sama. Apa masalahnya?

3. Mengapa StandardScaler harus di-fit di dalam tiap fold CV, bukan pada seluruh data?

4. Model fraud punya accuracy 99,8% dan recall 0%. Metrik apa yang akan Anda laporkan, dan mengapa?

5. Mengapa SMOTE tidak boleh diterapkan pada validation fold maupun test set?

RINGKASAN PERTEMUAN 4

- Ensemble → Bagging/RF menurunkan variance, Boosting menurunkan bias, Stacking memadukan
- Validasi → train/validation/test terpisah, Cross-Validation, tuning, tanpa data leakage
- Pipeline → semua `.fit()` (scaler, seleksi fitur, SMOTE) berada di dalam fold
- Imbalanced → accuracy menyesatkan; pakai Precision, Recall, F1, PR-AUC
- Penanganan → class weight, threshold moving, resampling/SMOTE (urutannya)

Terima Kasih - Q & A

PROJECT MILESTONE

Pertemuan 4: Mengembangkan Model dan Meningkatkan Akurasi

1	**Tinjau baseline Pertemuan 3**	Gunakan baseline dan evaluasi awal kelompok sebagai angka pembanding.
2	**Tetapkan strategi validasi**	Tentukan split/CV yang sesuai (Stratified, Group, Time Series) dan pastikan tidak ada leakage.
3	**Kembangkan model**	Coba minimal satu model ensemble (Random Forest atau Boosting) dan bandingkan dengan baseline.
4	**Lakukan tuning**	Setel hyperparameter dengan CV pada data latih saja.
5	**Periksa keseimbangan kelas**	Jika imbalanced: pilih metrik yang tepat dan coba class_weight / threshold / resampling di dalam Pipeline.

OUTPUT PROJECT PERTEMUAN 4

A. Strategi Validasi

- Skema split / CV yang dipilih;
- Alasan pemilihan;
- Bukti tidak ada leakage.

B. Model Ensemble

- Model yang dicoba;
- Hyperparameter dan hasil tuning;
- Perbandingan dengan baseline.

C. Penanganan Imbalance (jika ada)

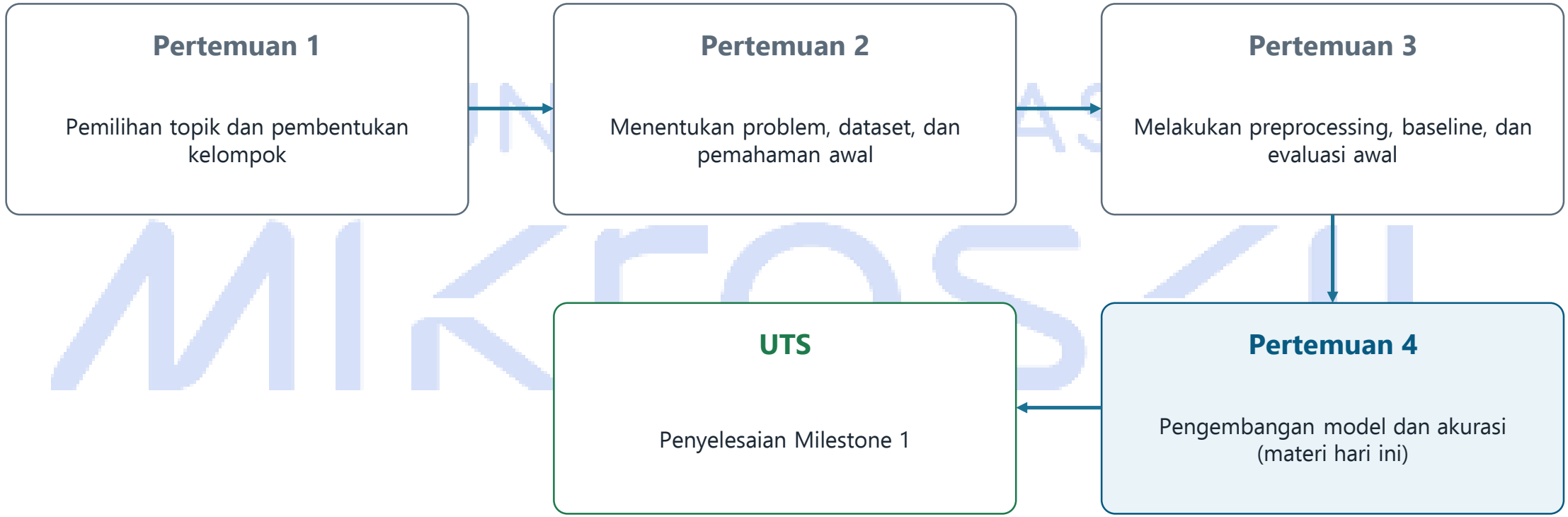
- Rasio kelas;
- Metrik yang dipakai;
- Teknik yang dicoba dan hasilnya.

D. Evaluasi & Temuan

- Tabel hasil (rata-rata ± simpangan baku);
- Analisis kesalahan;
- Kelemahan dan rencana perbaikan.

HUBUNGAN DENGAN UTS

Project Milestone



Pada UTS, proyek harus sudah menunjukkan bahwa proyek benar-benar berjalan, bukan hanya proposal.

REFERENSI

Bacaan Lanjutan

Topik	Sumber
Buku utama	A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow (bab Ensemble Learning; bab Training Models)
Bagging & Random Forest	L. Breiman (1996) Bagging Predictors; L. Breiman (2001) Random Forests, Machine Learning 45
Boosting	Freund & Schapire (1997) AdaBoost; J. Friedman (2001) Greedy Function Approximation; Chen & Guestrin (2016) XGBoost
Stacking	D. Wolpert (1992) Stacked Generalization
Validasi & leakage	Hastie, Tibshirani, Friedman, The Elements of Statistical Learning (bab 7); Bergstra & Bengio (2012) Random Search for Hyper-Parameter Optimization
Imbalanced data	Chawla et al. (2002) SMOTE; He & Garcia (2009) Learning from Imbalanced Data; Saito & Rehmsmeier (2015) PR vs ROC
Dokumentasi	scikit-learn: Ensemble methods, Cross-validation, Tuning hyper-parameters; imbalanced-learn user guide